



Simulacro de Examen — Spring Framework y Spring Boot (20 preguntas)

1. ¿Qué clase se utiliza para cargar beans definidos en un archivo XML?
 - A) `ApplicationContext`
 - B) `BeanFactory`
 - C) `ConfigurableBeanFactory`
 - D) `ComponentScan`
2. ¿Qué anotación se usa para indicar que una clase es un controlador REST?
 - A) `@Controller`
 - B) `@RestService`
 - C) `@RestController`
 - D) `@ApiController`
3. ¿Cuál es la anotación que permite inyectar una dependencia automáticamente en Spring?
 - A) `@Inject`
 - B) `@Bean`
 - C) `@Autowired`
 - D) `@Service`
4. ¿Qué clase gestiona el ciclo de vida de los beans en Spring?
 - A) `BeanDefinition`
 - B) `BeanFactoryPostProcessor`
 - C) `ApplicationContext`
 - D) `BeanWrapper`
5. ¿Qué anotación define un método que devuelve un bean manualmente?
 - A) `@Component`
 - B) `@Service`
 - C) `@Bean`
 - D) `@Create`
6. ¿Qué ocurre si dos beans del mismo tipo están disponibles para inyectar sin especificar un nombre?
 - A) Se lanza `DuplicateBeanException`
 - B) Spring elige el primero alfabéticamente
 - C) Se lanza `NoUniqueBeanDefinitionException`
 - D) Se resuelve automáticamente

7. ¿Qué anotación define una clase como un componente del DAO?
- A) @Dao
 - B) @Repository
 - C) @Service
 - D) @Entity
8. ¿Qué anotación se usa en Spring Boot para marcar la clase principal de arranque?
- A) @Main
 - B) @SpringBootTest
 - C) @SpringBootApplication
 - D) @AppRunner
9. ¿Cuál es la anotación que permite escanear paquetes automáticamente?
- A) @EnableAutoConfig
 - B) @ComponentScan
 - C) @ScanPackages
 - D) @AutoScan
10. ¿Qué anotación se puede usar para limitar un bean a una sola instancia por clase?
- A) @Prototype
 - B) @Scope("singleton")
 - C) @SingletonBean
 - D) @Instance(1)



Salidas de pantalla:

11. ¿Qué imprime este fragmento si hay un @Autowired sin bean disponible?

java

```
@Autowired private Servicio servicio;
```

- A) null
- B) Lanza NullPointerException
- C) Lanza NoSuchBeanDefinitionException
- D) Lanza UnsatisfiedDependencyException

12. ¿Qué imprime el siguiente código?

java

```
@Configuration public class AppConfig { @Bean public String saludo() { return "Hola Spring"; } }
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
System.out.println(ctx.getBean("saludo"));
```

A) Hola Spring

B) saludo

C) null

D) Error en tiempo de ejecución

13. ¿Qué ocurre si un bean es definido dos veces con el mismo ID en XML?

A) Spring usa el último

B) Se lanza una excepción

C) Se sobrescriben

D) Se ignoran ambos

14. ¿Qué hace `@Qualifier` en Spring?

A) Define una anotación personalizada

B) Indica el bean por nombre para inyección

C) Crea un nuevo bean

D) Filtra el log

15. ¿Qué anotación

combina `@Configuration`, `@EnableAutoConfiguration` y `@ComponentScan`?

A) `@SpringApp`

B) `@SpringMain`

C) `@SpringBootApplication`

D) `@EnableConfigScan`

16. ¿Qué anota un método para ejecutarse justo después de la creación del bean?

A) `@Init`

B) `@PostConstruct`

C) `@AfterBeanCreate`

D) `@OnLoad`

17. ¿Qué ocurre si se anota un método con `@Bean` dentro de una clase sin `@Configuration`?

A) No se registra como bean

B) Se ejecuta igual

C) Lanza excepción

D) Se convierte en singleton automáticamente

18. ¿Qué anotación define una clase como componente de negocio?

A) `@Repository`

B) `@Service`

C) `@Controller`

D) `@Bean`

19. ¿Qué método se usa para lanzar una aplicación Spring Boot?

A) `Spring.start()`

B) `BootApplication.run()`

C) `SpringApplication.run()`

D) `Main.run()`

20. ¿Qué ocurre si no se cierra un `ApplicationContext` manual?

A) Se cierra automáticamente

B) Puede causar pérdida de memoria o recursos

C) No afecta

D) Spring lanza excepción



RESPUESTAS Y EXPLICACIONES

1. A – `ApplicationContext` carga beans desde XML y ofrece funcionalidades avanzadas.
2. C – `@RestController` combina `@Controller` + `@ResponseBody`.
3. C – `@Autowired` inyecta dependencias automáticamente.
4. C – `ApplicationContext` gestiona el ciclo de vida completo de los beans.
5. C – `@Bean` define un método productor de beans dentro de `@Configuration`.
6. C – Spring lanza `NoUniqueBeanDefinitionException` si no puede elegir un bean único.
7. B – `@Repository` marca una clase como DAO y traduce excepciones.
8. C – `@SpringBootApplication` es la anotación de arranque.
9. B – `@ComponentScan` indica qué paquetes escanear.
10. B – `@Scope("singleton")` es el valor por defecto, pero también se puede especificar.
11. D – Si no hay bean, se lanza `UnsatisfiedDependencyException`.
12. A – El bean `saludo()` retorna "Hola Spring", que se imprime.
13. B – Spring lanza error si hay dos beans con el mismo ID.
14. B – `@Qualifier` selecciona un bean por nombre en inyecciones ambiguas.
15. C – `@SpringBootApplication` agrupa otras 3 anotaciones fundamentales.
16. B – `@PostConstruct` ejecuta un método justo tras crear el bean.
17. A – Sin `@Configuration`, `@Bean` no tiene efecto.
18. B – `@Service` define la lógica de negocio (capa de servicios).

19. **C** – `SpringApplication.run()` lanza una app Spring Boot.

20. **B** – No cerrar el contexto puede dejar recursos abiertos.

¿Quieres que este test se expanda a los 100 ejercicios o te genero otro test de 20 más de repaso diferente?

dime lo qye son estas anotaciones.....

@RestController
@Autowired
que es dao
) @Dao
B) @Repository
C) @Service
D) @Entity
@ComponentScan
@Qualifier

Claro, Borja. Aquí tienes una **explicación clara y concisa** de cada anotación y concepto que has mencionado, con su finalidad y uso habitual en Spring Framework y Spring Boot:

✓ @RestController

- **Qué es:** Una anotación compuesta que incluye `@Controller` y `@ResponseBody`.
- **Para qué sirve:** Indica que la clase maneja **peticiones REST** (HTTP) y que los métodos devuelven directamente datos (normalmente JSON) al cliente.
- **Ejemplo típico:**

```
java
@RestController public class MiControlador { @GetMapping("/saludo") public
String saludar() { return "Hola mundo"; } }
```

✓ @Autowired

- **Qué es:** Anotación de **inyección de dependencias automática**.
- **Para qué sirve:** Le dice a Spring que debe **inyectar automáticamente un bean** en ese punto, sin que tengas que instanciarlo manualmente.
- **Ejemplo:**

```
java
@Service public class MiServicio { @Autowired private RepositorioUsuario
repositorio; }
```

? ¿Qué es DAO?

- **Significa:** *Data Access Object*.
- **Qué hace:** Es una **capa de abstracción** encargada de **acceder a la base de datos** (consultas, inserciones, actualizaciones...).

- **Ejemplo:** Un DAO sería la clase que tiene métodos como `guardarUsuario()` , `buscarPorId()` , etc.



¿Qué anotación es adecuada para un DAO?

- **Correcta: B) `@Repository`**
- **Motivo:** Aunque `@Dao` existe como patrón de diseño, en Spring la anotación oficial para marcar clases DAO es `@Repository` .
- Spring gestiona automáticamente **la traducción de excepciones de base de datos** si usas `@Repository` .



`@ComponentScan`

- **Qué es:** Anotación que **indica a Spring qué paquetes escanear** para encontrar clases con anotaciones como `@Component` , `@Service` , `@Controller` , etc.
- **Para qué sirve:** Activar la **detección automática de beans**.
- **Ejemplo:**

```
java
@ComponentScan(basePackages = "com.miempresa.servicios")
```

Copiar código



`@Qualifier`

- **Qué es:** Anotación que se usa junto a `@Autowired` .
- **Para qué sirve:** Especifica **cuál bean inyectar** cuando hay **más de un bean del mismo tipo**.
- **Ejemplo:**

```
java
@Autowired @Qualifier("repositorioPostgres") private RepositorioUsuario
repositorio;
```

Copiar código